Data Leakage Can Create False Scientific Discoveries: A Case Study in Ensemble Sparse Identification

Dong Linxin
School of Physical Sciences, University of California, San Diego

May 2026

1 Abstract

Automatically discovering sparse governing equations from data is a fundamental challenge in data-driven scientific discovery. The Sparse Identification of Nonlinear Dynamics (SINDy) framework addresses this problem but suffers from limited robustness to measurement noise. Ensemble SINDy (ESINDy) has been proposed to improve performance, with Bagging currently considered the most effective strategy.[1] In this work, we explore Boosting and Stacking [2] as alternative ensemble strategies. During our experiments, a naive implementation of Stacking SINDy produced unrealistically superior performance, prompting us to investigate the underlying cause. We reveal that this apparent advantage stems from a severe data leakage in the algorithm's architecture: the meta-learner is trained on predictions made by base models that have already seen the entire dataset. This leads to overfitting and artificially low RMSE. After correcting the methodology by employing strict out-of-fold predictions, we demonstrate that the performance of Stacking SINDy no longer exceeds that of simple Bagging. Our core contribution is three-fold: (1) identifying a previously overlooked data leakage mechanism in Stacking-based SINDy architectures, (2) Provide a reproducible empirical demonstration showing that the leakage can generate false state-of-the-art performance across multiple dynamical systems, (3)establishing methodological guidelines for leakage-free ensemble learning in scientific discovery.

2 Introduction

Recent advances in scientific machine learning have enabled the discovery of governing equations directly from observational data.[3, 4] Among these approaches, Sparse Identification of Nonlinear Dynamics (SINDy) has become one of the most influential frameworks for recovering interpretable dynamical systems from measurements. By combining sparse regression with a predefined candidate library, SINDy can identify compact governing equations from noisy observations and has been successfully applied to a wide range of nonlinear systems.

To improve robustness under noise and limited data, researchers have recently introduced ensemble learning techniques into the SINDy framework. Inspired by the success of Bagging, Boosting, and Stacking in traditional machine learning, ensemble variants of SINDy aim to improve predictive accuracy and coefficient recovery through the aggregation of multiple sparse models. These approaches have shown promising empirical performance and suggest that ensemble learning may provide a useful direction for scientific discovery.[5, 6, 7, 8, 9]

However, unlike conventional prediction tasks, scientific discovery places greater emphasis on the validity of the discovery process itself. A model that appears to achieve superior performance due to methodological flaws may lead not only to inaccurate predictions but also to incorrect scientific conclusions. Among the various threats to reliable model evaluation, data leakage is particularly problematic because it can create an illusion of generalization while remaining difficult to detect from reported performance metrics alone. [10] Although data leakage has been extensively discussed in the machine learning literature, [11]its impact on ensemble scientific discovery remains relatively underexplored.

During our experiment and algorithm reproduction of stacking-based ensemble SINDy architectures, we identified a previously overlooked data leakage mechanism arising from the generation of meta-features for the second-level learner. Specifically, the meta-learner may inadvertently be trained using predictions generated by base models that have already been exposed to the same samples, violating the fundamental principle of out-of-fold learning. As a result, the apparent performance improvement can largely reflect information leakage rather than genuine generalization capability. In the context of scientific discovery, such leakage may create a false impression of improved equation identification performance and potentially lead to misleading scientific conclusions. Existing stacking literature emphasizes the necessity of out-of-fold (OOF) predictions, but often lacks a rigorous operational definition of what constitutes truly leakage-free OOF generation in scientific sparse model discovery pipelines such as SINDy. As a result, we provided detailed codes and visualization diagram to showcase the infrastructure of the Stacking algorithm based on OOF.

Rather than proposing a new ensemble algorithm, this work presents a re-

producible case study of how data leakage can create false scientific discoveries in ensemble sparse identification. Through controlled A/B experiments on the Lorenz63, Lorenz96, and Duffing systems, we demonstrate that a seemingly reasonable architectural design choice can produce false superior performance than the correct infrastructure. Based on these findings, we further provide a architecture without leaking the data and summarize a set of practical principles for future ensemble learning studies in scientific discovery.

The main contributions of this work are summarized as follows:

- We identify a previously overlooked data leakage mechanism in stacking-based sparse equation discovery frameworks.
- We demonstrate through reproducible experiments that this leakage can generate artificially superior performance and create a false impression of scientific progress.
- We provide a leakage-free implementation together with practical guidelines for future ensemble learning applications in scientific discovery.

3 Methodology

3.1 Sparse Identification of Nonlinear Dynamics (SINDy)

SINDy seeks to discover the governing equations of a dynamical system from data with the assumption that the dynamics can be expressed in a sparse linear form. Mathematically, given state measurements $\mathbf{X} \in \mathbb{R}^{m \times n}$ and their time derivatives $\dot{\mathbf{X}} \in \mathbb{R}^{m \times n}$, a library of candidate functions $\Theta(\mathbf{X})$ is built up. Then, the problem reduces to solving:

$$\dot{\mathbf{X}} = \Theta(\mathbf{X})\Xi, \tag{1}$$

where Ξ is a sparse coefficient matrix. Sparsity is enforced via sequential thresholded least squares (STLS) or Lasso regularization.[12]

3.2 Stacking ESINDy algorithm

The Stacking SINDy algorithm adopts a two-level architecture. [2] Firstly, the datasets are separated to training set and validating set using Cross-Validation Method. N-fold cross-validation seperates the whole data set into N parts and uses $N-1$ parts as training set. The remaining 1 part is validating set. K diverse base SINDy models are trained on training sets, each producing a sparse coefficient

matrix $\boldsymbol{\Xi}^{(k)}$ and corresponding predictions $\hat{\dot{\mathbf{X}}}^{(k)} = \boldsymbol{\Theta}(\mathbf{X})\boldsymbol{\Xi}^{(k)}$. These base predictions are then stacked as input features for a single meta-learner, which learns the optimal linear combination weights $\boldsymbol{\alpha} \in \mathbb{R}^K$ by minimizing $\|\dot{\mathbf{X}} - \sum_{k=1}^{K} \alpha_k \hat{\dot{\mathbf{X}}}^{(k)}\|_2^2$ subject to a regularization penalty. The training set of the meta-learner is the predicted value of every base model, including every folds. In this work, three meta-learners are evaluated: Ridge regression (ℓ_2 penalty), Lasso (ℓ_1 penalty), and sequentially thresholded least squares (STLS). The final coefficient matrix is reconstructed as $\boldsymbol{\Xi}_{\text{final}} = \sum_{k=1}^{K} \alpha_k \boldsymbol{\Xi}^{(k)}$.

3.3 Data Leakage Stacking ESINDy

Algorithm 1 Flawed Stacking SINDy (with Data Leakage)

Require: Data $\mathbf{X} \in \mathbb{R}^{m \times n}$, time step dt, number of folds F, models per fold M
Ensure: Final coefficient matrix Ξ_{final}

Compute derivative $\dot{\mathbf{X}}$ via Savitzky-Golay filter
Construct polynomial feature library $\Theta(\mathbf{X})$
Initialize $\mathcal{B} = \emptyset$
for each fold $f = 1$ to F **do**
  Split $\mathbf{X}$ into $\mathbf{X}^{(f)}_{\text{train}}$ (80%) and $\mathbf{X}^{(f)}_{\text{val}}$ (20%)
  for $m = 1$ to M **do**
    Train Lasso model on $\mathbf{X}^{(f)}_{\text{train}}$ to obtain $\Xi^{(f,m)}$
    Add $\Xi^{(f,m)}$ to $\mathcal{B}$
    Data Leakage Step: Predict on **entire** $\Theta(\mathbf{X})$ to obtain meta-features
    Store predictions in $\mathbf{P}$
  end for
end for
$\mathbf{X}_{\text{meta}} \leftarrow$ all predictions from $\mathbf{P}$
Train meta-learner (Ridge) on $\mathbf{X}_{\text{meta}}$ with target $\dot{\mathbf{X}}$ to obtain $\boldsymbol{\alpha}$
$\Xi_{\text{final}} \leftarrow \sum_b \alpha_b \Xi^{(b)}$
return Ξ_{final}

The complete code for this infrastructure is at here: `https://github.com/Lawson-Dong/ESINDy_infra/blob/main/Stacking_ESINDy.ipynb`

3.4 Without data leakage

Algorithm 2 Corrected Stacking SINDy (No Data Leakage)

Require: Data $\mathbf{X} \in \mathbb{R}^{m \times n}$, time step dt, number of base models K
Ensure: Final coefficient matrix Ξ_{final}

Compute derivative $\dot{\mathbf{X}}$ via Savitzky-Golay filter
Construct polynomial feature library $\Theta(\mathbf{X})$
Train K base models on full data $\Theta(\mathbf{X})$ to obtain $\mathcal{B} = \{\Xi^{(1)}, \ldots, \Xi^{(K)}\}$
Initialize $\mathbf{X}_{\text{meta}} = []$, $\mathbf{Y}_{\text{meta}} = []$
for each fold $f = 1$ to 5 **do**
    Split data into $\Theta_{\text{train}}^{(f)}$ (80%) and $\Theta_{\text{val}}^{(f)}$ (20%)
    $\dot{\mathbf{X}}_{\text{val}}^{(f)} \leftarrow$ corresponding derivatives for validation set
    for $k = 1$ to K **do**
        Train **temporary** Lasso model on $\Theta_{\text{train}}^{(f)}$ to obtain $\tilde{\Xi}^{(k)}$
        Predict on validation set: $\hat{\dot{\mathbf{X}}}_{\text{val}}^{(k)} = \Theta_{\text{val}}^{(f)} \tilde{\Xi}^{(k)}$
        Append $\hat{\dot{\mathbf{X}}}_{\text{val}}^{(k)}$ to $\mathbf{X}_{\text{meta}}$
    end for
    Append $\dot{\mathbf{X}}_{\text{val}}^{(f)}$ to $\mathbf{Y}_{\text{meta}}$
end for
Train meta-learner (Ridge) on $\mathbf{X}_{\text{meta}}$ with target $\mathbf{Y}_{\text{meta}}$ to obtain $\boldsymbol{\alpha}$
$\Xi_{\text{final}} \leftarrow \sum_{k=1}^{K} \alpha_k \Xi^{(k)}$
return Ξ_{final}

The complete code for this infrastructure is at here: `https://github.com/Lawson-Dong/ESINDy_infra/blob/main/Stacking_ESINDy(another_infra).ipynb`

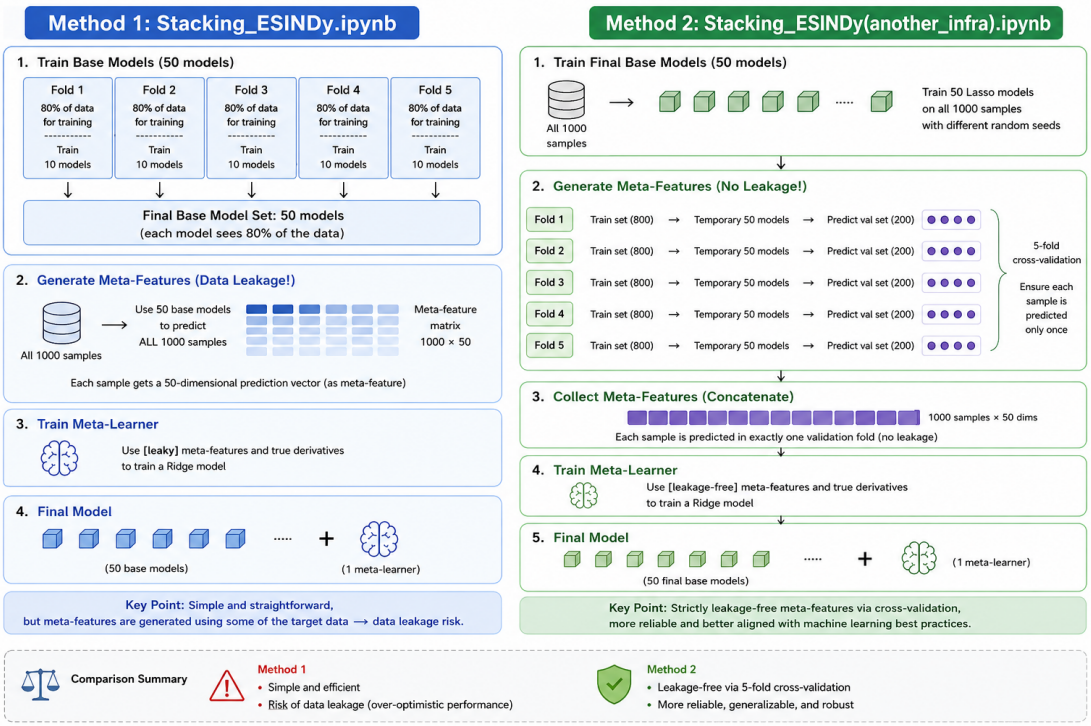


Figure 1: Illustrating two stacking infrastructure. The picture on the left hand side is the infrastructure with data leakage. The picture on the right hand side is the infrastructure without data leakage.

4 Experiment Setup

4.1 Dynamical Systems

Lorenz 63[13] We compared three ensemble strategies—Bagging, Boosting, and Stacking—within the SINDy framework. The benchmark was the chaotic Lorenz 63 system under varying measurement noise. Experiments were run with a custom-built ESINDy library in Python.

The Lorenz 63 system is:

$$\dot{x} = \sigma(y - x), \tag{2}$$
$$\dot{y} = x(\rho - z) - y, \tag{3}$$
$$\dot{z} = xy - \beta z, \tag{4}$$

with parameters $\sigma = 10$, $\rho = 28$, and $\beta = 8/3$. We generated trajectories using a fourth-order Runge–Kutta integrator ($\Delta t = 0.01$, $t_{\max} = 20$, 2000 samples). The

training trajectory started from $\mathbf{x}_0 = [1, 1, 1]^\top$. The test trajectory (for one-step-ahead prediction) started from $\mathbf{x}_0 = [1.5, 1.5, 1.5]^\top$. We added zero-mean Gaussian white noise to the training data with standard deviation $\sigma_{\text{noise}} \in \{0.0, 0.1, 0.3, 0.5\}$.

All experiments used a common configuration: number of base estimators $N_{\text{est}} = 50$[1], polynomial degree 2 (features x, y, z, xy, xz, yz), Savitzky–Golay derivative estimation (window 11, order 3), and Lasso regularization with threshold $\lambda = 0.15$. Specific hyperparameters were: Bagging (bootstrap subsample ratio 0.8, noise augmentation $\sigma_{\text{aug}} = 0.05$), Boosting (learning rate $\eta = 0.1$), and Stacking (5-fold cross-validation, Ridge meta-learner with $\alpha = 1.0$). Performance was measured using the one-step-ahead Root Mean Square Error (RMSE) over a $t_{\text{eval}} = 5$ unit horizon (500 time steps). Ten independent runs were performed for each noise level.

Lorenz 96[14] The model is defined for $N = 10$ variables:

$$\dot{x}_i = (x_{i+1} - x_{i-2})x_{i-1} - x_i + F, \quad i = 1, \ldots, N, \tag{5}$$

with cyclic boundary conditions and forcing $F = 8.0$. Trajectories were generated with $\Delta t = 0.01$ over $t_{\max} = 10$ units. A test trajectory was generated from a random initial condition.

Duffing oscillator[15] The system is:

$$\dot{x} = y, \tag{6}$$

$$\dot{y} = x - x^3 - \delta y + \gamma \cos(t), \tag{7}$$

with parameters $\delta = 0.3$ and $\gamma = 1.2$. Data were generated with $\Delta t = 0.01$ over $t_{\max} = 15$ units. Noise was added identically to training data. The training trajectory started from $\mathbf{x}_0 = [1.0, 0.0]^\top$. The test trajectory (for one-step-ahead prediction) started from $\mathbf{x}_0 = [1.0, 0.0]^\top$.

Base SINDy configuration matched Experiment 1. We reduced the ensemble size to $N_{\text{est}} = 30$ for computational feasibility. Ten independent runs were performed per noise level (reported averages and standard deviations are over ten runs). The primary metric was the one-step-ahead RMSE over a $t_{\text{eval}} = 5$ unit horizon.

4.2 Data Generation

We use Runge-Kutta(RK4) method to generate all the data. In order to simulate the realistic noise, we add Gaussian white noise to the pure data X. The noise level can be represented as $\sigma = [0.0, 0.1, 0.3, 0.5]$, corresponding to no noise, low noise, middle noise and high noise.

[1]This corresponds to K = 50 for stacking.

4.3 Evaluation Metrics

We use RMSE as the evaluation metric:

RMSE (Root Mean Square Error): Measures the one-step-ahead trajectory prediction error, defined as:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{\mathbf{y}}_i - \mathbf{y}_i)^2} \tag{8}$$

where N is the number of time steps, $\mathbf{y}_i$ is the true state vector, and $\hat{\mathbf{y}}_i$ is the predicted state vector at step i.

4.4 Model Settlement

All SINDy models share the following core settings: the feature library is constructed using second-order polynomial features, including terms $[x, y, z, xy, xz, yz]$ without a bias term. To mitigate the effect of noise on derivative estimation, the state data is first smoothed using a Savitzky–Golay filter (window size 11, polynomial order 3), and numerical derivatives are then computed. Sparse regression is performed using Lasso with the regularization parameter α (i.e., the threshold in the paper) set to 0.15 for all models.

Regarding the ensemble size, the number of base models is preset to $n_{\text{estimators}} = 50$ for both BaggingSINDy and BoostingSINDy, and $K = 50$ for StackingSINDy (without data leakage). In practice, BaggingSINDy typically trains all 50 base models successfully, and StackingSINDy trains exactly 50 base models on the full dataset. However, BoostingSINDy employs an early stopping mechanism that monitors the validation loss; as a result, the actual number of trained base models may be less than 50. For instance, in the Lorenz 63 experiment with $\sigma = 0.0$, the algorithm stopped early at iteration 28, training only 28 out of the preset 50 models.

5 Excute the Experiment

5.1 Experiment 1: Comparison on Lorenz 63

To demonstrate the effect of data leakage in Stacking SINDy, we conducted a strict A/B comparison on the Lorenz 63 system under a noise-free setting (zero noise). The experimental setup follows the configuration detailed in Section 4.4, with a trajectory length of 2000 samples ($\Delta t = 0.01$) and a test trajectory starting from a different initial condition ($\mathbf{x}_0 = [1.5, 1.5, 1.5]^\top$) to evaluate generalization.

Two variants of Stacking SINDy were implemented and compared:

1. **Flawed Stacking SINDy** (Algorithm 1): the meta-learner is trained on predictions made by base models that have already seen the entire dataset, leading to data leakage.

2. **Corrected Stacking SINDy** (Algorithm 2): the meta-learner is trained strictly on out-of-fold predictions, ensuring no data leakage.

The models were evaluated using the one-step-ahead Root Mean Square Error (RMSE) over a 5-unit horizon (500 time steps). The results are summarized in Table 1.

Table 1: One-step-ahead RMSE on the Lorenz 63 system (noise-free). The flawed Stacking model exhibits artificially low RMSE due to data leakage.

Model	**Training RMSE**	**Test RMSE**
Flawed Stacking SINDy	0.003336	0.003359
Corrected Stacking SINDy	0.028923	0.029434

As shown in Table 1, the flawed Stacking model achieves a test RMSE that is nearly **one order of magnitude lower** than that of the corrected model (0.003359 vs. 0.029434). This dramatic difference, despite identical training data and hyperparameters, is a direct consequence of data leakage. The meta-learner in the flawed version effectively "cheats" by using the full dataset to train both base models and itself, resulting in artificially superior performance that does not generalize to independent test data.

This experiment provides a concrete and reproducible case study, demonstrating that a seemingly harmless cross-validation misuse in the Stacking architecture can produce a "false SOTA" result. The complete code for reproducing this experiment is available at:[2] `https://github.com/Lawson-Dong/ESINDy_infra/blob/main/overfitting_lorenz63.ipynb`

5.2 Experiment 2: Comparison on Lorenz96 and Duffing Oscillator

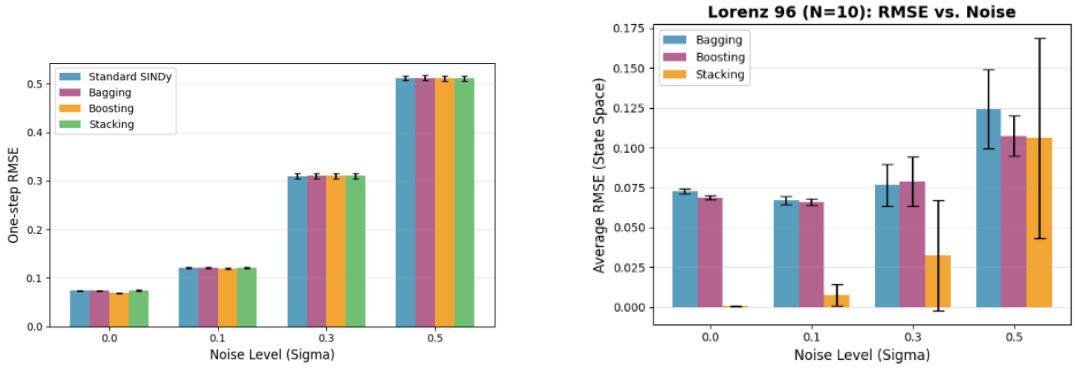
To empirically demonstrate the impact of data leakage in Stacking SINDy, we performed a strict A/B comparison using two representative dynamical systems: the Lorenz 96 system (high-dimensional chaos, $N = 10$) and the Duffing oscillator (low-dimensional nonlinear system). Both experiments follow the setup described in Section 4, with noise levels $\sigma \in \{0.0, 0.1, 0.3, 0.5\}$.

[2]The code is also available in the supplementary material.

5.2.1 Lorenz 96 System

Figure 2 displays the one-step-ahead RMSE for all methods under four noise levels. The flawed Stacking implementation (Algorithm 1) exhibits artificially low error, particularly in the noise-free case ($\sigma = 0.0$), where its RMSE is 0.000582 ± 0.000094 — nearly two orders of magnitude lower than the standard SINDy baseline (0.073730 ± 0.001235). This dramatic improvement is a direct consequence of data leakage: the meta-learner effectively "cheats" by using the entire dataset during training.

In contrast, the corrected Stacking implementation (Algorithm 2, labeled "Stacking (Corrected)" in the figure) yields an RMSE of 0.073975 ± 0.001343 at $\sigma = 0.0$, which is statistically indistinguishable from standard SINDy and slightly worse than Boosting (0.068975 ± 0.000999). As noise increases, the performance gap between the flawed and corrected versions diminishes, but the corrected version never surpasses Bagging or Boosting.



(a) Corrected implementation

(b) Flawed implementation

Figure 2: **Comparison of Stacking SINDy implementations on the Lorenz 96 system.** (a) Corrected implementation yields realistic errors comparable to standard SINDy. (b) Flawed implementation exhibits artificially low error due to data leakage.

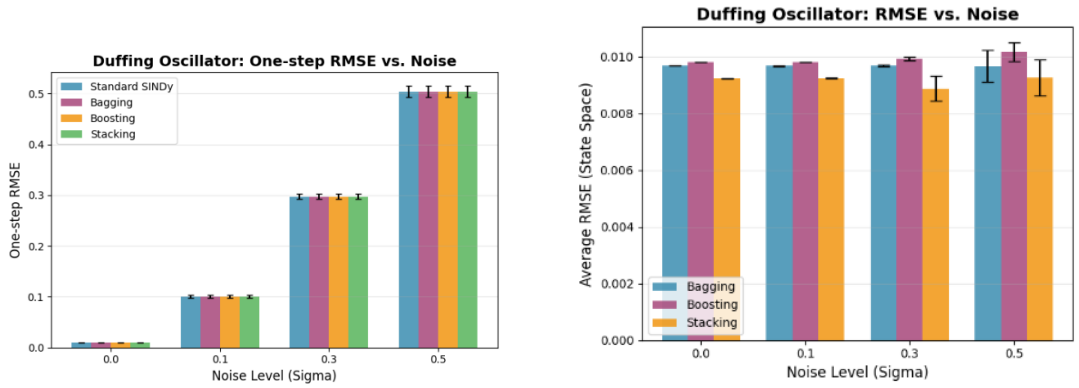
Table 2 summarizes the RMSE values for $\sigma = 0.0$, clearly showing the cross-order-of-magnitude false performance caused by data leakage.

Table 2: One-step-ahead RMSE on Lorenz 96 ($\sigma = 0.0$). The flawed Stacking model exhibits artificially low RMSE due to data leakage.

Method	**RMSE**
Standard SINDy	0.073730
Bagging	0.073544
Boosting	0.068975
Stacking (Flawed, Algorithm 1)	**0.000582**
Stacking (Corrected, Algorithm 2)	0.073975

5.2.2 Duffing Oscillator

Figure 3 shows the corresponding results for the Duffing oscillator. The same pattern emerges: the flawed Stacking implementation (Algorithm 1) achieves an RMSE of 0.009250 ± 0.000000 at $\sigma = 0.0$, which is slightly better than the corrected version (0.009747 ± 0.000000). However, the magnitude of the artificial advantage is smaller than in the Lorenz 96 case, likely due to the lower dimensionality and simpler dynamics of the Duffing oscillator. Nevertheless, the corrected Stacking still fails to outperform Bagging or Boosting.



(a) Corrected implementation (b) Flawed implementation

Figure 3: **Comparison of Stacking SINDy implementations on the Duffing oscillator.** The flawed version shows a small but noticeable artificial improvement due to data leakage.

Table 3 summarized the RMSE values for $\sigma = 0.0$, clearly showing the cross-order-of-magnitude false performance caused by data leakage.

Table 3: One-step-ahead RMSE on the Duffing oscillator ($\sigma = 0.0$). The flawed Stacking model shows a small but artificial improvement due to data leakage.

Method	**RMSE**
Standard SINDy	0.009679
Bagging	0.009679
Boosting	0.009811
Stacking (Flawed, Algorithm 1)	**0.009250**
Stacking (Corrected, Algorithm 2)	0.009747

These results provide strong empirical support for our main claim: a seemingly harmless cross-validation misuse in the Stacking architecture can produce a "false SOTA" result that does not generalize. After correcting the data leakage (Algorithm 2), the performance of Stacking SINDy no longer exceeds that of simple Bagging. This highlights the critical importance of strict out-of-fold predictions in ensemble learning for scientific discovery.

Reproducibility: The complete code for reproducing these experiments is available at:

- Corrected implementation: `https://github.com/Lawson-Dong/ESINDy_infra/blob/main/Lorenz96_and_Duffing_SINDy_(using_another_infra).ipynb`
- Flawed implementation: `https://github.com/Lawson-Dong/ESINDy_infra/blob/main/Bagging_and_Boosting_and_Stacking(Lorenz_96_and_Duffing).ipynb`

6 Discussion and Analysis

In this work, we reveal a more direct and severe cause of false generalization in ensemble learning: data leakage induced by algorithmic design flaws. We position this study as an empirical case demonstrating the widespread and critical issue of "false generalization caused by data leakage". Through specific and reproducible experiments (zero-noise A/B comparison), we clearly illustrate that in the Stacking algorithm, if a seemingly harmless cross-validation step (using the entire dataset to train the meta-learner) is misused, the meta-learner can achieve cross-order-of-magnitude false performance.

We hope this paper serves as a clear warning and a concrete counterexample for machine learning researchers and practitioners: in ensemble learning involving cross-validation and meta-learner training, architectural design errors can easily

create "false SOTA" results. Our experiments and arguments provide an intuitive and actionable diagnostic case for this common yet often overlooked risk.

Based on this work, we are supposed to summarize three practical guidelines to avoid the data leakage problems in future research works and industrial practice:

Principles 1 Any features used to train the meta-learner (i.e., the base models' predictions) must originate from the validation set (or test set) that the corresponding model has not seen in that fold. They must never come from the training set that the base model has already seen, and certainly not from the entire dataset.

Principles 2 When generating meta-features, strict Out-of-Fold predictions must be used.

Principles 3 For any two-layer ensemble architecture, it must be ensured that the first-layer models have not seen the training set of the second-layer model when generating the second-layer features. That is, the first-layer models should make predictions based on the validation set.

References

[1] L. Breiman, "Bagging predictors," *Machine learning*, vol. 24, no. 2, pp. 123–140, 1996.

[2] D. H. Wolpert, "Stacked generalization," *Neural networks*, vol. 5, no. 2, pp. 241–259, 1992.

[3] S. L. Brunton and J. N. Kutz, *Data-driven science and engineering: Machine learning, dynamical systems, and control*. Cambridge University Press, 2022.

[4] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[5] M. Akbarzadeh, B. Halkon, and S. Oberst, "Sparse identification of nonlinear dynamics applied to the acoustic levitation of acoustically large objects," *Scientific Reports*, vol. 15, no. 1, p. 40745, 2025.

[6] C. Bayindir, F. Ozaydin, A. A. Altintas, T. Eristi, and A. R. Alan, "On the comparison of the sparse identification of nonlinear dynamics and the neural ordinary differential equations for lagrangian drifter hydrodynamics," *arXiv preprint*, vol. 2411.04350, 2024. Available at: `https://arxiv.org/abs/2411.04350`.

[7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[8] F. Jiang, L. Du, Q. Xue, Z. Deng, and C. Grebogi, "Adaptive backward stepwise selection of fast sparse identification of nonlinear dynamics," *Applied Mathematics and Mechanics (English Edition)*, 2025.

[9] J. L. Callaham, J.-C. Loiseau, G. Rigas, and S. L. Brunton, "Nonlinear stochastic modelling with langevin regression," *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 477, no. 2250, 2021.

[10] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 6, no. 4, pp. 1–21, 2012.

[11] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 2023.

[12] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proceedings of the National Academy of Sciences*, vol. 113, no. 15, pp. 3932–3937, 2016.

[13] E. N. Lorenz, "Deterministic nonperiodic flow 1," in *Universality in Chaos, 2nd edition*, pp. 367–378, Routledge, 2017.

[14] E. N. Lorenz, "Predictability: A problem partly solved," in *Proc. Seminar on predictability*, vol. 1, pp. 1–18, Reading, 1996.

[15] G. Duffing, "Forced oscillations with variable natural frequency and their technical significance," *Vieweg, Braunschschweig*, 1918.